

DIALOGUE NATIONAL

MÉMO TECHNIQUE DE LA PLATEFORME NUMÉRIQUE

dialoguenational.cd

Version 1.0

15 septembre 2026

Document de transmission technique et institutionnelle.

Établi à partir de l'inspection du dépôt de code, de la base de données et du service en ligne.

Ne contient aucun identifiant, mot de passe, jeton ni chaîne de connexion.

Avertissement de lecture

Ce mémo décrit l'état du dépôt de code à la date indiquée, commit 92fd1ac sur la branche main du dépôt github.com/harnolmub/dialogue. Chaque élément a été vérifié dans le code, la configuration, la base de données de développement ou les en-têtes HTTP du site en production.

La mention **À confirmer** signale une information qui ne peut pas être établie depuis le dépôt : elle relève des consoles d'administration (Railway, Neon, Cloudflare, bureau d'enregistrement du domaine) auxquelles seul le détenteur des accès peut répondre. Ces mentions ne sont pas des oublis : elles indiquent précisément ce que l'équipe entrante doit se faire transmettre.

Aucun mot de passe, jeton, clé ni chaîne de connexion ne figure dans ce document.

1. PRÉSENTATION DU PROJET

1.1 Objet de la plateforme

`dialoguenational.cd` est la plateforme numérique de recueil des contributions citoyennes destinées aux travaux du Dialogue national de la République démocratique du Congo. Elle remplit deux fonctions distinctes :

1. **Un site public** qui explique la démarche et permet à toute personne de déposer une contribution, sans création de compte et sans pièce justificative.
2. **Un backoffice** réservé aux équipes habilitées, qui reçoit, classe, annote et synthétise ces contributions.

1.2 Participation citoyenne et diaspora

Le dépôt est ouvert à toute personne, en République démocratique du Congo comme à l'étranger. Le formulaire demande un pays de résidence choisi dans la liste complète des pays reconnus ; lorsque le pays sélectionné est la RDC, un champ « province » apparaît. Les contributions déposées depuis un pays autre que la RDC sont comptabilisées comme contributions de la diaspora et font l'objet d'une vue dédiée dans le backoffice.

1.3 Formats acceptés

Une contribution doit comporter **au moins un** des trois formats, et peut les combiner :

Format	Limite technique appliquée
Texte	20 000 caractères
Note vocale	4 minutes, un seul enregistrement
Documents	5 fichiers, 10 Mo par fichier, formats PDF, DOC, DOCX, TXT, JPG, PNG

Un envoi ne comportant aucun de ces trois formats est refusé avec un message explicite.

1.4 Traitement administratif

Chaque dépôt reçoit une référence unique de la forme `DIALOGUE-2026-000000`, affichée à l'écran immédiatement après l'envoi. Cette référence est la seule preuve de dépôt remise au contributeur : aucun courriel de confirmation n'est envoyé à ce jour (voir section 2.9).

Les contributions ne sont jamais publiées automatiquement. Elles entrent dans un circuit de traitement à statuts, décrit en section 8, entièrement piloté depuis le backoffice.

1.5 Portée de l'outil

La plateforme est un **outil d'appui au processus**. Elle collecte, organise et restitue ; elle ne décide pas. La recevabilité d'une contribution, sa qualification, son importance et son intégration aux travaux relèvent de décisions humaines prises par les équipes habilitées. Aucune automatisation du dépôt au classement n'est en place aujourd'hui, et l'architecture proposée en section 9 pour l'intelligence artificielle maintient explicitement ce principe.

2. ARCHITECTURE TECHNIQUE

2.1 Vue d'ensemble

L'application est un **serveur Node.js unique** qui sert à la fois les pages statiques compilées et l'API. Il n'y a ni service séparé, ni file d'attente, ni tâche planifiée, ni microservice.

2.2 Frontend

Élément	Valeur vérifiée
Framework	Aucun. JavaScript natif, sans React, Vue, Svelte ni équivalent
Langage	JavaScript ES modules, pas de TypeScript
Outil de compilation	Vite 7 (^7.1.7), configuration multi-pages
Pages compilées	index.html, participer.html, admin.html, mentions.html, 404.html
Styles	CSS écrit à la main : style.css, identity.css, participer.css, admin.css, countries.css, voice.css
Polices	Public Sans et Geist Mono, chargées depuis Google Fonts
Dépendances front au moment de l'exécution	Aucune bibliothèque tierce n'est chargée dans le navigateur, hors les polices

Le backoffice (admin.html + src/admin.js) est une application d'une seule page écrite en JavaScript natif, avec un routage par fragment d'URL (#/contributions, #/syntheses/<id>, etc.).

2.3 Backend

Élément	Valeur vérifiée
Exécution	Node.js — version non figée dans package.json (pas de champ engines)
Framework	Express 5 (^5.1.0)
Validation des entrées	Zod (^3.24.2)
Réception de fichiers	Multer (^1.4.5-lts.2), stockage en mémoire puis écriture
Limitation de débit	express-rate-limit (^7.5.0)
Client PostgreSQL	pg (^8.14.1)
Client de stockage objet	@aws-sdk/client-s3 (^3.750.0)

Le serveur applique deux en-têtes de sécurité à toutes les réponses : X-Frame-Options: DENY et Permissions-Policy: microphone=(self).

2.4 Base de données

PostgreSQL, accédé en **SQL direct**. Il n'y a **aucun ORM** : les requêtes sont écrites à la main dans `server/db.js` et `server/admin-api.js`, toutes paramétrées.

L'hébergeur de la base est **Neon** (base PostgreSQL gérée, connexion via le point d'entrée « pooler », SSL exigé).
Détail en section 4.

Un **mode de repli** existe : si la variable `DATABASE_URL` est absente au démarrage, le serveur écrit les contributions dans un fichier local `data/contributions.json` et journalise un avertissement. Ce mode est destiné au développement ; il ne doit jamais servir en production, où il entraînerait une perte de données à chaque redémarrage.

2.5 Authentification

Implémentation propre au projet, sans bibliothèque tierce d'authentification :

- mots de passe hachés avec **scrypt** (sel aléatoire de 16 octets, empreinte de 64 octets), comparaison à temps constant ;
- session matérialisée par un jeton aléatoire de 32 octets, transmis dans un cookie `admin_session` (`httpOnly`, `SameSite=Strict`, `Secure` en production) et stocké en base **sous forme de condensat SHA-256** — le jeton en clair n'existe que dans le navigateur ;
- durée de session : 12 heures par défaut, réglable par variable d'environnement ;
- jeton anti-CSRF distinct, exigé dans l'en-tête `x-csrf-token` sur toute écriture ;
- quatre rôles avec permissions explicites (section 7.2).

2.6 Stockage des fichiers et des notes vocales

Un seul module, `server/storage.js`, avec deux comportements selon la configuration :

Condition	Comportement
Les quatre variables <code>S3_BUCKET</code> , <code>S3_ENDPOINT</code> , <code>S3_ACCESS_KEY_ID</code> , <code>S3_SECRET_ACCESS_KEY</code> sont définies	Envoi vers un stockage objet compatible S3
Sinon	Écriture sur le disque local, sous <code>data/uploads/</code>

Les fichiers sont rangés sous la clé `contributions/<identifiant>/audio|documents/<uuid>.<ext>`. Ils ne sont **jamais** exposés publiquement : leur lecture passe obligatoirement par une route d'administration authentifiée qui diffuse le flux (`Cache-Control: private, no-store`).

À confirmer : le stockage effectivement utilisé en production. Le fichier `.env` local ne définit aucune variable `S3_*`, ce qui signifierait un stockage disque — or le disque d'un conteneur Railway est éphémère sauf volume persistant attaché. C'est le point de vigilance le plus important de ce mémo : voir sections 3.6, 11.4 et 17.3.

2.7 API

Une seule route publique d'écriture, `POST /api/contributions`, et vingt-cinq routes d'administration sous `/api/admin/`. Liste complète en section 7.8.

2.8 Intelligence artificielle

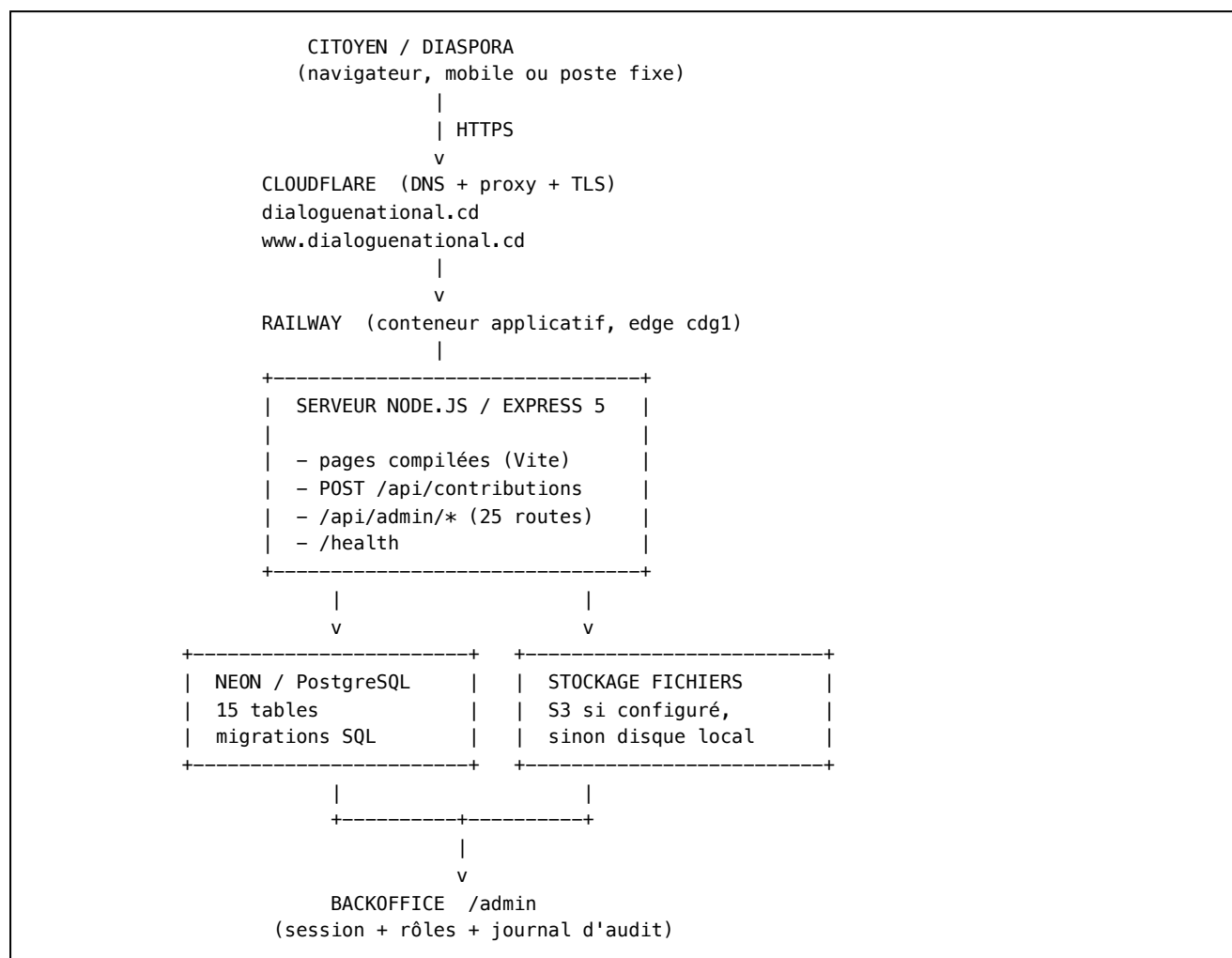
Aucune. Aucune dépendance, clé d'API, appel réseau ni code lié à un fournisseur d'IA n'existe dans le dépôt. L'assistant présent sur la page d'accueil est un **questionnaire scripté** de cinq questions à choix multiples, entièrement codé en dur dans `src/main.js` : il n'appelle aucun service et ne produit aucun texte. La section 9 décrit l'architecture cible.

2.9 Services externes

Service	État vérifié
Analytique (Google Analytics, Plausible, Matomo...)	Aucun
Envoi de courriels	Aucun — aucune bibliothèque, aucun appel SMTP ou API
Supervision / rapport d'erreurs (Sentry...)	Aucun
Polices	Google Fonts (dépendance externe au moment du rendu)
Stockage objet S3	Code présent, activation à confirmer

Conséquence directe : **le contributeur ne reçoit aucun courriel** et l'équipe n'est alertée d'aucune panne par un outil automatique. Voir sections 14 et 15.

2.10 Schéma de l'architecture réelle



Ce qui **n'existe pas** dans cette chaîne : file d'attente, service de transcription, service d'envoi de courriels, moteur de recherche, service d'IA, tâche planifiée, second service applicatif.

3. HÉBERGEMENT

3.1 Éléments vérifiés depuis l'extérieur

Les en-têtes HTTP de `https://dialoguenational.cd/` établissent avec certitude :

Indice observé	Conclusion
<code>server: cloudflare, cf-ray</code> , serveurs de noms <code>*.ns.cloudflare.com</code>	Cloudflare assure le DNS et sert de proxy
Enregistrements A pointant sur <code>104.21.77.86</code> et <code>172.67.205.192</code>	Adresses Cloudflare : le trafic est bien relayé (proxy activé)
<code>x-railway-request-id, x-railway-edge: cdg1</code>	L'application est hébergée sur Railway , edge de Paris
<code>x-powered-by: Express</code>	Le serveur Express répond bien derrière le proxy
<code>http://</code> répond 301 vers <code>https://</code>	Redirection en place
<code>www.dialoguenational.cd</code> répond 200	Le sous-domaine sert l'application (il ne redirige pas)

3.2 Ce que le dépôt ne contient pas

Il n'existe **aucun fichier de configuration d'hébergement** : ni `railway.json`, ni `railway.toml`, ni `nixpacks.toml`, ni Procfile, ni Dockerfile. La configuration du service est donc entièrement portée par l'interface Railway.

À confirmer auprès du détenteur du compte Railway :

- nom du projet et nom du service ;
- commande de compilation et commande de démarrage configurées ;
- version de Node.js retenue par la plateforme ;
- branche déclenchant le déploiement (la pratique observée est `main`, à valider) ;
- présence ou non d'un volume persistant, et son point de montage ;
- liste des variables d'environnement définies ;
- existence d'un environnement de préproduction.

3.3 Fonctionnement du déploiement

Le dépôt n'embarquant pas la configuration, la chaîne probable est le comportement par défaut de Railway : détection automatique du projet Node.js, exécution de `npm install`, puis du script `build` s'il existe, puis du script `start`. Ce projet fournit bien les deux :

- `npm run build` → `vite build`, qui produit le dossier `dist/` ;
- `npm start` → `node --env-file-if-exists=.env server.js`.

Ce point est **critique** : `dist/` est exclu du dépôt par `.gitignore`. Si la commande de compilation n'est pas exécutée au déploiement, le serveur démarre sans pages à servir et répond en erreur sur toutes les routes HTML.

À confirmer dans les réglages du service.

Au démarrage, le serveur exécute dans l'ordre :

1. `ensureDatabase()` — création de la table `schema_migrations` si absente, puis application de toutes les migrations `.sql` non encore enregistrées, par ordre alphabétique de nom de fichier ;
2. `bootstrapAdmin()` — création du compte super administrateur si `ADMIN_BOOTSTRAP_EMAIL` et `ADMIN_BOOTSTRAP_PASSWORD` sont définis et que ce courriel n'existe pas encore ;
3. écoute sur le port fourni par la variable `PORT`.

Si l'initialisation de la base échoue, le processus s'arrête avec un code d'erreur — le service ne démarre pas dans un état incohérent.

3.4 Exploitation courante sur Railway

Opération	Marche à suivre
Consulter les journaux	Console Railway → projet → service → onglet des journaux (build et exécution séparés)
Redémarrer	Console Railway → service → redéploiement du déploiement courant
Revenir en arrière	Console Railway → historique des déploiements → redéployer un déploiement antérieur
Vérifier la santé	<code>curl https://dialoguenational.cd/health</code> doit renvoyer <code>{"ok":true}</code>

La route `/health` ne teste que la vivacité du processus HTTP ; elle n'interroge pas la base. Une base injoignable après le démarrage renverra malgré tout `{"ok":true}`. Voir section 15.

3.5 Domaines

Domaine	État vérifié
<code>dialoguenational.cd</code>	Actif, HTTPS, sert l'application
<code>www.dialoguenational.cd</code>	Actif, HTTPS, sert l'application (pas de redirection vers le domaine nu)
<code>http://dialoguenational.cd</code>	Redirige en 301 vers HTTPS

Le fichier `public/sitemap.xml` et les balises canoniques des pages désignent le domaine **sans `www`** comme adresse de référence. Le fait que `www` serve le même contenu sans redirection crée un contenu dupliqué du point de vue du référencement. Correction recommandée : une règle de redirection Cloudflare de `www` vers le domaine nu.

À confirmer : bureau d'enregistrement du domaine `.cd`, date d'expiration, titulaire déclaré, et détenteur des accès. Aucun enregistrement MX n'est publié : **aucune adresse de courriel `@dialoguenational.cd` ne peut recevoir de message** en l'état.

3.6 Cloudflare

Vérifié : DNS délégué à Cloudflare, proxy actif, TLS terminé chez Cloudflare, en-têtes NEL et `cf-cache-status` : DYNAMIC sur les pages HTML.

À confirmer dans la console Cloudflare : mode de chiffrement TLS (doit être « Full (strict) »), activation de « Always Use HTTPS », règles de page ou de redirection, pare-feu applicatif et règles de limitation, activation du mode « Under Attack » en cas d'affluence, et paramètres de cache.

4. BASE DE DONNÉES

4.1 Nature et rôle

PostgreSQL géré par **Neon**. Les chaînes de connexion utilisées par le projet visent un point d'entrée « pooler » de Neon avec `sslmode=require`. Deux bases distinctes sont utilisées :

Environnement	Usage	État
Développement	Travail local et tests automatisés	Accessible, migrations 001, 002 et 003 appliquées
Production	Site en ligne	À confirmer — voir ci-dessous

Point d'attention : la chaîne de connexion de production transmise en cours de projet **n'est plus valide** (échec d'authentification pour l'utilisateur de la base, code 28P01). Le mot de passe a vraisemblablement été renouvelé, ou la branche Neon recréée. L'équipe entrante doit récupérer la chaîne courante depuis les variables du service Railway ou depuis la console Neon.

4.2 Connexion applicative

Un pool pg unique est créé au démarrage à partir de `DATABASE_URL`. En production (`NODE_ENV=production`), le client force `ssl: { rejectUnauthorized: false }`. Toutes les requêtes sont paramétrées ; aucune concaténation de valeur utilisateur dans du SQL n'a été relevée.

4.3 Tables (15 tables vérifiées sur la base de développement)

Table	Rôle	Colonnes remarquables
contributions	Dépôt citoyen — table centrale, 29 colonnes	reference (unique, générée), status, priority, assigned_to, theme, secondary_theme, text_content, audio_key, audio_duration, audio_mime, audio_size, transcription, detected_language, transcription_validated, audio_summary, internal_note, read_at
contribution_files	Documents joints	storage_key, original_name, mime_type, size, suppression en cascade
contribution_tags	Étiquettes posées sur une contribution	clé primaire composée
contribution_status_history	Historique des changements de statut	from_status, to_status, changed_by
contribution_assignments	Historique des affectations	assigned_to, assigned_by
internal_comments	Commentaires internes	body limité à 10 000 caractères
admin_users	Comptes du backoffice	email forcé en minuscules, password_hash, role, active, last_login_at
admin_sessions	Sessions ouvertes	token_hash (clé primaire), csrf_token, expires_at, last_seen_at
password_reset_tokens	Réinitialisation de mot de passe	Table créée mais inutilisée : aucune route ne l'alimente
admin_themes	Référentiel des thématiques	15 thématiques insérées par la migration
admin_tags	Référentiel des étiquettes	name, slug, color, active
summaries	Notes de synthèse	title, body, theme, status (DRAFT, REVIEW, PUBLISHED)
summary_contributions	Rattachement contributions ↔ synthèse	clé primaire composée
audit_logs	Journal des actions du backoffice	action, resource_type, resource_id, details (JSONB), ip_hash
schema_migrations	Migrations déjà appliquées	name, applied_at

4.4 Relations principales

```
admin_users --< contributions.assigned_to
admin_users --< internal_comments.author_id
admin_users --< audit_logs.user_id
admin_users --< summaries.created_by

contributions --< contribution_files          (cascade)
contributions --< contribution_tags >-- admin_tags
contributions --< internal_comments          (cascade)
contributions --< contribution_status_history (cascade)
contributions --< contribution_assignments   (cascade)
contributions --< summary_contributions >-- summaries
```

La suppression d'une contribution efface donc ses fichiers, commentaires, historique, affectations et étiquettes. La suppression d'un compte administrateur ne supprime rien : les références sont mises à nul, l'historique reste lisible.

4.5 Migrations

Fichiers SQL numérotés dans migrations/, appliqués automatiquement au démarrage du serveur et enregistrés dans schema_migrations.

Fichier	Contenu
001_contributions.sql	Table des contributions, fichiers joints, séquence de référence, statuts initiaux
002_admin_backoffice.sql	Statuts supplémentaires, colonnes de traitement, 12 tables du backoffice, index, 15 thématiques
003_summary_status.sql	Cycle de vie des synthèses

Application manuelle possible : `npm run migrate`. Le script échoue explicitement si `DATABASE_URL` est absente.

Règle à respecter : ne jamais modifier une migration déjà appliquée ; toujours en ajouter une nouvelle. Le mécanisme ne gère pas les retours en arrière — un correctif se fait par une migration supplémentaire.

4.6 Sauvegardes et restauration

Le dépôt ne contient **aucun script de sauvegarde**. La sauvegarde repose donc entièrement sur les fonctions de Neon (historique de branche et restauration à un instant donné).

À confirmer dans la console Neon : durée de rétention de l'historique, procédure de restauration retenue, existence d'exports réguliers hors Neon, et responsable de leur vérification. Tant que ce point n'est pas établi, la plateforme n'a **pas** de garantie de sauvegarde vérifiée.

4.7 Accès sécurisé

Connexion chiffrée exigée. La chaîne de connexion est un secret : elle ne doit exister que dans les variables d'environnement du service et, localement, dans un fichier `.env` — lequel est exclu du dépôt par `.gitignore` (règles `.env` et `.env.*`, avec exception pour `.env.example`).

5. FONCTIONNEMENT DU SITE PUBLIC

5.1 Pages servies

Adresse	Contenu
/	Accueil : présentation, repères, thématiques, documents, assistant guidé
/participer	Formulaire de dépôt
/mentions	Mentions légales, traitement des données, accessibilité
/admin	Backoffice (page de connexion si aucune session)
/health	État du service, réponse JSON
toute autre adresse	Page 404 dédiée, avec le code HTTP 404

5.2 Parcours du contributeur

Accueil

- > Comprendre le Dialogue (sections d'information, modale « Comment se passe un dépôt »)
- > Éventuellement, assistant guidé (5 questions) qui prépare le choix de la thématique
- > Page « Participer »
 - Étape 01 : vos informations
 - Étape 02 : objet de la contribution
 - Étape 03 : votre contribution (texte, note vocale, documents)
 - Consentement obligatoire
- > Validation dans le navigateur
- > Envoi au serveur, nouvelle validation
- > Enregistrement en base et écriture des fichiers
- > Écran de confirmation avec la référence de dépôt

5.3 Champs du formulaire

Champ	Obligatoire	Contrainte
Prénom	Oui	80 caractères
Nom	Oui	80 caractères
Adresse électronique	Oui	format vérifié, 254 caractères
Téléphone ou WhatsApp	Non	40 caractères
Pays de résidence	Oui	liste ISO complète, RDC par défaut
Ville ou territoire	Oui	100 caractères
Province	Non	affichée seulement si le pays est la RDC
Qualité du contributeur	Non	liste fermée
Thématique	Oui	15 thématiques
Titre de la proposition	Non	180 caractères
Texte	Selon	20 000 caractères
Note vocale	Selon	4 minutes
Documents	Selon	5 × 10 Mo
Consentement	Oui	case à cocher

« Selon » signifie qu'au moins un des trois formats doit être fourni.

5.4 Consentement et données personnelles

La case de consentement est obligatoire et son libellé précise que les contributions ne sont pas publiées automatiquement et que les extraits repris dans les synthèses sont anonymisés. Côté serveur, la valeur on est exigée : un envoi sans consentement est refusé, quel que soit le reste.

La page /mentions détaille la finalité, les destinataires, la durée de conservation et les droits d'accès, de rectification et de suppression, exercés au moyen de la référence de dépôt.

5.5 Validation et messages d'erreur

La validation est faite **deux fois** : dans le navigateur pour le confort, sur le serveur pour la sécurité. Les messages sont rédigés en français, sans jargon technique.

Situation	Code HTTP	Message affiché
Champ obligatoire manquant	400	« Certaines informations obligatoires sont manquantes ou incorrectes. »
Consentement absent	400	« Vous devez accepter le traitement de votre contribution. »
Aucun format fourni	400	« Ajoutez au moins un format : un texte, une note vocale ou un document. »
Note vocale trop longue	400	« La note vocale doit durer 4 minutes au maximum. »
Format de fichier refusé	415	« Le format de <i>nom du fichier</i> n'est pas accepté. »
Fichier trop volumineux	413	« Chaque fichier doit rester sous 10 Mo. »
Trop d'envois	429	« Trop de tentatives depuis cet appareil. Réessayez dans quelques minutes. »
Erreur serveur	500	« Votre contribution n'a pas pu être enregistrée. Réessayez dans quelques instants. »

Dans le navigateur, chaque champ fautif est signalé individuellement, le premier champ en erreur reçoit le focus, et un résumé apparaît au-dessus du bouton d'envoi.

6. FONCTIONNEMENT DES NOTES VOCALES

C'est la partie la plus délicate de la plateforme : elle dépend du navigateur, du système et des autorisations de l'appareil. Le code correspondant se trouve dans `src/participer.js`.

6.1 Chaîne technique

Étape	Mise en œuvre vérifiée
Détection des capacités	<code>navigator.mediaDevices?.getUserMedia</code> et <code>window.MediaRecorder</code> testés au chargement
Demande d'accès	<code>getUserMedia({ audio: { echoCancellation: true, noiseSuppression: true } })</code>
Choix du format	Premier format accepté parmi <code>audio/mp4</code> , <code>audio/webm; codecs=opus</code> , <code>audio/webm, audio/ogg; codecs=opus</code>
Capture	<code>MediaRecorder</code> , fragments collectés dans un tableau
Niveau sonore	<code>AudioContext + AnalyserNode</code> , jauge de 12 barres rafraîchie en continu
Assemblage	<code>new Blob(fragments, { type })</code> , le type étant lu sur les fragments eux-mêmes
Prévisualisation	<code>URL.createObjectURL</code> sur un lecteur <code><audio controls></code>
Contrôle du silence	Décodage du fichier produit et mesure de l'amplitude maximale
Envoi	Ajouté au <code>FormData</code> sous le champ <code>audio</code> , avec la durée mesurée

6.2 Parcours de l'utilisateur

```
Bouton « Démarrer l'enregistrement »
-> Demande d'autorisation du microphone
    refus ou échec -> message d'aide ciblé + possibilité d'importer un fichier audio
-> Enregistrement en cours
    jauge de niveau visible, minuterie, pause et reprise possibles
    arrêt automatique à 4 minutes de parole effective (les pauses ne comptent pas)
-> « Terminer »
-> Finalisation : attente du dernier fragment avant de constituer le fichier
-> Prévisualisation : l'utilisateur réécoute
    si le fichier est silencieux, un avertissement explicite s'affiche
-> « Supprimer et recommencer » possible à tout moment
-> Envoi du formulaire : la note est transmise avec la contribution
-> Association en base : colonnes audio_key, audio_duration, audio_mime, audio_size
```

6.3 Particularités par navigateur

Navigateur	Comportement traité dans le code
Chrome / Android	Format WebM Opus, cas nominal
Safari / iPhone	audio/mp4 testé en premier ; le type du fragment fait foi car <code>recorder.mimeType</code> peut être vide ; la finalisation attend explicitement le dernier fragment, livré tardivement sur mobile
Navigateur sans MediaRecorder	Le bouton d'enregistrement est masqué et l'import d'un fichier audio est proposé à la place
Contexte non sécurisé (HTTP)	<code>getUserMedia</code> est indisponible : message invitant à ouvrir la page en HTTPS

Le repli par import de fichier accepte les formats audio/webm, mp4, mpeg, mp3, ogg, wav, x-m4a et aac, avec la même limite de 4 minutes, vérifiée par décodage réel du fichier.

6.4 Dépannage microphone

Le code distingue et explique sept erreurs. Table de correspondance pour le support :

Erreur technique	Ce que voit l'utilisateur	Cause et résolution
<code>NotAllowedError</code>	« Microphone non autorisé »	L'autorisation a été refusée. Réglages du navigateur → site → autoriser le microphone, puis « Réessayer »
<code>NotFoundError</code>	« Microphone introuvable »	Aucun périphérique d'entrée. Brancher ou activer un micro
<code>NotReadableError</code>	« Microphone indisponible »	Le micro est pris par une autre application (appel en cours). Fermer l'application concernée
<code>AbortError</code>	« Enregistrement interrompu »	Interruption système. Réessayer
<code>SecurityError</code>	« Accès sécurisé requis »	Page ouverte en navigation privée, en iframe ou hors HTTPS
<code>OverconstrainedError</code>	« Configuration audio incompatible »	Contraintes audio non satisfaites par l'appareil
<code>TypeError</code>	« Microphone indisponible »	Contexte non sécurisé : vérifier que l'adresse commence par <code>https://</code>

Cas « la jauge ne bouge pas » : si aucun son n'est capté pendant l'enregistrement, un avertissement apparaît au bout de quelques secondes, et le fichier produit est analysé après coup. Un fichier silencieux déclenche le message « La note enregistrée est silencieuse », en nommant l'entrée microphone utilisée. La cause est presque toujours une entrée coupée ou mal sélectionnée dans les réglages du système.

Contrainte d'infrastructure : l'en-tête `Permissions-Policy: microphone=(self)` n'autorise le microphone que pour l'origine du site. Une intégration du formulaire dans une iframe d'un autre site empêcherait l'enregistrement — comportement voulu.

7. BACKOFFICE

7.1 Accès

Adresse : <https://dialoguenational.cd/admin>

La page est exclue de l'indexation (robots en `noindex, nofollow`). Elle affiche un écran de connexion tant qu'aucune session valide n'est présente. Il n'existe **pas** de formulaire de mot de passe oublié : la réinitialisation est faite par un super administrateur (section 7.7).

7.2 Rôles et permissions

Rôle	Lire	Écrire sur une contribution	Statistiques	Fichiers	Exports	Synthèses	Étiquettes	Comptes et audit
SUPER_ADMIN	oui	oui	oui	oui	oui	oui	oui	oui
ADMIN	oui	oui	oui	oui	oui	oui	oui	non
ANALYST	oui	oui	non	oui	non	oui	oui	non
VIEWER	oui	non	non	non	non	non	non	non

Les permissions sont vérifiées **côté serveur** sur chaque route, pas seulement dans l'interface.

7.3 Tableau de bord

À quoi ça sert : mesurer l'activité et repérer ce qui attend un traitement.

Comment l'utiliser : huit indicateurs cliquables — total, non consultées, à traiter, en traitement, validées, non assignées, notes vocales, diaspora. Un clic ouvre la liste des contributions déjà filtrée. En dessous : dépôts des trente derniers jours, répartition par thématique, huit derniers dépôts, provinces les plus actives.

7.4 Liste des contributions

À quoi ça sert : retrouver une contribution et traiter un lot.

Comment l'utiliser : une barre de filtres — recherche libre (référence, nom, courriel, contenu), statut, thématique, pays, format, priorité, personne assignée, non consultées. Les colonnes Référence, Date, Statut et Priorité sont triables. La pagination affiche 25 lignes par défaut. Les contributions jamais ouvertes sont marquées d'un liseré.

Actions groupées : cocher plusieurs lignes fait apparaître une barre permettant d'appliquer un statut ou une affectation à l'ensemble, en une opération (200 éléments au maximum).

Export : le bouton « Exporter en CSV » respecte les filtres actifs. Colonnes : Référence, Date, Prénom, Nom, Email, Pays, Province, Profil, Thématique, Statut, Priorité. Fichier encodé en UTF-8 avec marque d'ordre des octets, directement lisible dans un tableur. Limite : 10 000 lignes.

7.5 Fiche d'une contribution

À quoi ça sert : tout ce qui concerne un dépôt, au même endroit.

Contenu : identité et coordonnées du contributeur, origine, thématique, titre ; texte intégral ; lecteur audio ; documents joints ; commentaires internes ; historique des statuts.

Comment l'utiliser :

- **Écouter la note vocale** — le lecteur diffuse le fichier par une route authentifiée ; l'écoute est journalisée.
- **Télécharger un document** — lien nominatif, téléchargement journalisé.
- **Transcrire** — deux champs libres, « Transcription » et « Langue détectée », saisis à la main aujourd'hui (voir section 9).
- **Changer le statut** — liste des huit statuts ; tout changement est historisé avec son auteur.
- **Fixer la priorité** — Normale, Suivie, Haute, Urgente.
- **Affecter** — à un compte actif du backoffice ; l'affectation est historisée.
- **Note interne** — champ libre, distinct des commentaires.
- **Étiqueter** — cases à cocher, et création d'une étiquette depuis la fiche.
- **Commenter** — fil de commentaires internes horodatés et signés.

L'ouverture d'une fiche marque la contribution comme consultée (read_at).

7.6 Analyse et diaspora

À quoi ça sert : lire la participation par géographie et par sujet.

Comment l'utiliser : tableau des contributions de la diaspora par pays, avec le nombre de contributions et le nombre de contributeurs distincts ; poids relatif des thématiques.

7.7 Synthèses, comptes et audit

Synthèses — création d'une note (titre, thématique, contenu), puis modification avec un cycle de vie Brouillon → En relecture → Publiée. Des contributions peuvent être rattachées à une synthèse. La synthèse est **rédigée par un analyste** ; aucune génération automatique n'existe.

Comptes (super administrateur) — liste des comptes avec rôle, dernière connexion et état ; changement de rôle ; activation et désactivation ; création d'un compte avec mot de passe provisoire d'au moins 16 caractères. Un super administrateur ne peut ni se désactiver ni se rétrograder lui-même. Toute désactivation ou changement de mot de passe **supprime immédiatement les sessions ouvertes** du compte concerné.

Journal d'audit (super administrateur) — quinze types d'actions enregistrées : LOGIN, LOGIN_FAILED, LOGOUT, CONTRIBUTION_UPDATED, CONTRIBUTIONS_BULK_UPDATED, COMMENT_ADDED, TAGS_UPDATED, TAG_CREATED, AUDIO_STREAMED, FILE_DOWNLOADED, EXPORT_CSV, SUMMARY_CREATED, SUMMARY_UPDATED, USER_CREATED, USER_UPDATED. Filtrage par action, pagination. L'adresse IP n'est **pas** conservée en clair : seul son condensat est stocké.

7.8 Routes d'administration

Méthode et route	Permission requise
POST /api/admin/auth/login	— (limitée à 5 tentatives / 15 min / IP)
POST /api/admin/auth/logout	session valide
GET /api/admin/me	session valide
GET /api/admin/dashboard	analytics:read
GET /api/admin/filters	contributions:read
GET /api/admin/contributions	contributions:read
GET /api/admin/contributions/:id	contributions:read
PATCH /api/admin/contributions/:id	contributions:write
POST /api/admin/contributions/bulk	contributions:write
POST /api/admin/contributions/:id/comments	contributions:write
PUT /api/admin/contributions/:id/tags	tags:write
GET /api/admin/contributions/:id/audio	files:read
GET /api/admin/contributions/:id/files/:fileId	files:read
GET et POST /api/admin/tags	contributions:read / tags:write
GET /api/admin/analysis	analytics:read
GET, POST, GET /:id, PATCH /:id sur /api/admin/summaries	summaries:write
GET, POST /api/admin/users, PATCH /api/admin/users/:id	super administrateur
GET /api/admin/audit	super administrateur
GET /api/admin/exports/contributions.csv	exports:run

7.9 Administration des documents officiels

Fonctionnalité absente du backoffice. Les documents officiels sont aujourd'hui des fichiers déposés dans `public/assets/` et référencés en dur dans la page d'accueil. Leur ajout passe donc par une modification du code et un déploiement. Le modèle proposé figure en section 10.

8. CYCLE DE TRAITEMENT D'UNE CONTRIBUTION

8.1 Statuts réellement définis

Le type `contribution_status` contient neuf valeurs. Huit sont proposées dans le backoffice ; la neuvième, `SELECTED`, subsiste de la première version du schéma et n'est plus utilisée par l'interface.

Statut	Libellé affiché	Sens
RECEIVED	Reçue	Statut initial de tout dépôt
IN_REVIEW	En examen	Prise en charge par un analyste
NEEDS_FOLLOW_UP	À compléter	Nécessite une précision du contributeur
VALIDATED	Validée	Retenue pour la synthèse
DUPLICATE	Doublon	Déjà reçue sous une autre référence
OUT_OF_SCOPE	Hors sujet	Hors du champ du Dialogue
REJECTED	Rejetée	Écartée (contenu irrecevable)
ARCHIVED	Archivée	Traitement clos
SELECTED	—	Hérité, non utilisé

8.2 Circuit

CITOYEN	
v	
Dépôt (site public)	référence DIALOGUE-AAAA-NNNNNN remise à l'écran
v	
Réception	statut RECEIVED, horodatage, fichiers stockés
v	
Backoffice	ouverture de la fiche -> marquée « consultée »
v	
Analyse	lecture, écoute, transcription éventuelle commentaires internes, note interne
v	
Classification	thématique, étiquettes, priorité, affectation
v	
Décision humaine	VALIDATED NEEDS_FOLLOW_UP DUPLICATE OUT_OF_SCOPE REJECTED
v	
Synthèse	rattachement à une note de synthèse DRAFT -> REVIEW -> PUBLISHED
v	
Restitution	travaux du Dialogue national

Chaque changement de statut est inscrit dans `contribution_status_history` avec son auteur, et chaque action dans `audit_logs`. Le circuit est **entièrement manuel** : aucune règle automatique ne fait passer une contribution d'un statut à un autre.

9. INTELLIGENCE ARTIFICIELLE

9.1 État actuel

Aucune brique d'IA n'est présente. Ni dépendance, ni clé, ni appel réseau. La section qui suit décrit une architecture **cible**, à construire. Deux colonnes de la base ont toutefois été prévues pour l'accueillir : transcription, detected_language, transcription_validated et audio_summary existent déjà dans la table contributions et sont aujourd'hui remplies à la main.

9.2 Principe directeur

L'IA est une **couche d'assistance**, jamais une dépendance du dépôt citoyen. Si le service d'IA est indisponible, la plateforme doit continuer à recevoir, stocker et afficher les contributions sans dégradation. Aucune sortie d'IA ne doit être visible du public sans validation humaine préalable.

9.3 A — Assistant documentaire public

```
Documents officiels validés
  -> découpage en passages
  -> calcul des représentations vectorielles
  -> index de recherche
  -> question d'un citoyen
  -> recherche des passages pertinents
  -> génération d'une réponse à partir de ces seuls passages
  -> réponse accompagnée de la citation et du lien vers le document source
```

Règles : répondre **uniquement** à partir des documents officiels marqués comme disponibles pour l'IA ; citer systématiquement la source ; répondre « je ne sais pas » plutôt que d'extrapoler ; ne jamais donner d'interprétation politique. Prérequis : la table des documents officiels décrite en section 10, qui n'existe pas encore.

9.4 B — IA administrative

```
Contribution reçue
  -> si note vocale : transcription automatique
  -> résumé en quelques lignes
  -> thématiques suggérées (jamais imposées)
  -> mots-clés
  -> rapprochement avec des contributions similaires (détection de doublons)
  -> proposition présentée à l'analyste
  -> validation, correction ou rejet par l'analyste
```

Toute sortie est un **brouillon**. Le champ transcription_validated existe précisément pour distinguer une transcription automatique d'une transcription vérifiée par un humain. Les suggestions doivent être stockées dans des champs distincts des données d'origine.

9.5 C — Synthèses assistées

Un analyste sélectionne un ensemble de contributions et demande une proposition de synthèse. Le texte produit arrive au statut DRAFT, doit être relu (REVIEW) puis publié (PUBLISHED) par une personne habilitée. Le cycle de vie existe déjà en base.

9.6 Interdits

L'IA ne doit **jamais** :

- supprimer ou archiver automatiquement une contribution ;
- juger seule de l'importance politique d'une proposition ;
- modifier le contenu original déposé par un citoyen ;
- publier une synthèse sans validation humaine ;
- prendre une décision institutionnelle ;
- être un passage obligé pour qu'une contribution soit enregistrée.

9.7 Conséquences techniques à prévoir

- Traitement **asynchrone** : le dépôt ne doit jamais attendre une réponse d'IA.
- Nouvelles variables d'environnement pour la clé du fournisseur, à traiter comme des secrets.
- Journalisation des appels dans `audit_logs` (modèle, coût, durée, succès ou échec).
- Encadrement du traitement de données personnelles : une contribution contient un nom et une adresse électronique ; leur transmission à un service tiers doit être décidée et documentée, et la mention de traitement de la page `/mentions` mise à jour en conséquence.

10. DOCUMENTS OFFICIELS

10.1 Situation actuelle

Un seul document est publié : l'allocution du Chef de l'État, fichier PDF placé dans `public/assets/` et lié en dur depuis la page d'accueil. **Il n'existe ni table, ni interface, ni métadonnées.** Ajouter un document impose aujourd'hui : déposer le fichier dans `public/assets/`, ajouter une ligne dans `index.html`, valider, pousser et déployer.

10.2 Procédure provisoire

1. Nommer le fichier en minuscules, sans accent ni espace (exemple : `ordonnance-2026-014.pdf`).
2. Le placer dans `public/assets/`.
3. Ajouter une entrée dans la liste « Documents et repères » de `index.html`, avec sa date, son intitulé, son type et son poids.
4. Compiler, tester, pousser sur main, vérifier après déploiement que le lien répond.

10.3 Modèle de métadonnées recommandé

À implémenter sous forme de table `official_documents` et d'un écran de gestion dans le backoffice :

Champ	Type	Rôle
<code>title</code>	texte	Intitulé officiel
<code>type</code>	liste	Discours, ordonnance, communiqué, termes de référence, rapport, document du Dialogue
<code>published_at</code>	date	Date de publication officielle
<code>source</code>	texte	Institution émettrice
<code>official_url</code>	texte	Adresse de la publication d'origine
<code>version</code>	texte	Version ou révision
<code>status</code>	liste	Brouillon, publié, retiré
<code>visibility</code>	liste	Public ou interne
<code>ai_indexable</code>	booléen	Le document peut-il alimenter l'assistant documentaire
<code>storage_key</code>	texte	Emplacement du fichier
<code>checksum</code>	texte	Empreinte, pour détecter une substitution

Le champ `ai_indexable` est la clé de voûte de la section 9.3 : seuls les documents explicitement marqués doivent être interrogeables par l'assistant public.

11. SÉCURITÉ

11.1 En place et vérifié

Mesure	Mise en œuvre
HTTPS	Cloudflare, redirection 301 depuis HTTP
Protection contre l'encadrement	X-Frame-Options: DENY sur toutes les réponses
Microphone restreint	Permissions-Policy: microphone=(self)
Mots de passe	script, sel aléatoire, comparaison à temps constant, 16 caractères minimum
Sessions	Jeton de 32 octets, stocké en base sous forme de condensat, cookie httpOnly + SameSite=Strict + Secure en production, expiration 12 h
Anti-CSRF	Jeton de session exigé dans l'en-tête sur toute écriture
Contrôle d'accès	Quatre rôles, permission vérifiée sur chaque route côté serveur
Validation des entrées	Zod sur toutes les entrées publiques et d'administration
Contrôle des fichiers	Type MIME sur liste blanche, refus explicite avec le nom du fichier
Taille maximale	10 Mo par fichier, 6 fichiers par envoi
Stockage privé	Aucun fichier accessible publiquement ; diffusion par route authentifiée en private, no-store
Traversée de répertoire	Clé de fichier validée (contributions/ obligatoire, .. refusé)
Limitation de débit	10 dépôts / 15 min / IP ; 5 connexions d'administration / 15 min / IP
Journal d'audit	15 types d'actions, IP conservée sous forme de condensat uniquement
Injection SQL	Requêtes intégralement paramétrées
Injection HTML	Échappement systématique des valeurs affichées dans le backoffice
Révocation	Désactivation d'un compte ou changement de mot de passe : sessions supprimées immédiatement
Secrets	.env et .env.* exclus du dépôt

11.2 Manques identifiés

Manque	Conséquence	Recommandation
Pas de Content-Security-Policy	Surface d'attaque plus large en cas d'injection	Ajouter un en-tête CSP restrictif
Pas de Strict-Transport-Security	Première visite en HTTP possible	Activer HSTS chez Cloudflare
Pas de réinitialisation de mot de passe	La table existe mais aucune route ne l'utilise	Implémenter, ou documenter la procédure manuelle par un super administrateur
Pas de second facteur	Un mot de passe suffit pour accéder à toutes les contributions	Envisager un second facteur pour les rôles élevés
Pas de vérification du contenu réel des fichiers	Le type MIME est déclaré par le client	Vérifier la signature binaire du fichier
Pas d'analyse antivirus	Un document infecté peut être stocké puis téléchargé par un agent	Analyser à la réception
Limitation par IP seulement	Contournable derrière un partage d'adresse	Compléter par une limitation applicative
Compte d'amorçage permanent	Le mot de passe initial reste valide tant qu'il n'est pas changé	Changer le mot de passe du compte d'amorçage après la première connexion
Sauvegardes non vérifiées	Restauration non testée	Voir section 4.6

11.3 Données personnelles

Chaque contribution contient un nom, un prénom, une adresse électronique, éventuellement un téléphone, une ville et une province. Ces données sont conservées en base sans chiffrement au niveau applicatif (le chiffrement au repos relève de Neon — **à confirmer**). L'export CSV les contient en clair : sa diffusion doit être encadrée, et chaque export est journalisé.

11.4 Risque principal

Si le stockage des fichiers est aujourd'hui sur disque local sans volume persistant, **chaque redéploiement efface les notes vocales et les documents déjà reçus**, alors que la base conserve leurs références — les fiches afficheraient des fichiers introuvables. Ce point doit être vérifié **en priorité** (section 3.2).

12. VARIABLES D'ENVIRONNEMENT

Noms uniquement. Aucune valeur ne figure dans ce document, et aucune ne doit être inscrite dans le dépôt.

Nom	Service concerné	Utilité	Obligatoire
DATABASE_URL	Neon / PostgreSQL	Chaîne de connexion à la base	Oui — sans elle, écriture dans un fichier local, non durable
PORT	Railway	Port d'écoute du serveur	Oui (fourni automatiquement par Railway)
NODE_ENV	Application	Active le SSL base et le cookie Secure	Oui — doit valoir production en ligne
ADMIN_BOOTSTRAP_EMAIL	Backoffice	Courriel du super administrateur créé au premier démarrage	Oui, au premier déploiement
ADMIN_BOOTSTRAP_PASSWORD	Backoffice	Mot de passe initial, 16 caractères minimum	Oui, au premier déploiement
ADMIN_SESSION_HOURS	Backoffice	Durée de session, 12 par défaut	Non
MAX_UPLOAD_BYTES	Dépôt	Taille maximale par fichier, 10 485 760 par défaut	Non
PUBLIC_RATE_LIMIT_MAX	Dépôt	Nombre de dépôts par IP et par quart d'heure, 10 par défaut	Non
ADMIN_LOGIN_RATE_LIMIT_MAX	Backoffice	Tentatives de connexion par IP et par quart d'heure, 5 par défaut	Non
S3_ENDPOINT	Stockage	Adresse du service de stockage objet	Oui si stockage S3
S3_BUCKET	Stockage	Nom du compartiment	Oui si stockage S3
S3_REGION	Stockage	Région, auto par défaut	Non
S3_ACCESS_KEY_ID	Stockage	Identifiant d'accès	Oui si stockage S3
S3_SECRET_ACCESS_KEY	Stockage	Clé secrète	Oui si stockage S3
S3_FORCE_PATH_STYLE	Stockage	Adressage par chemin	Non
VITE_API_TARGET	Développement	Cible du proxy du serveur de développement	Non
E2E_PORT	Tests	Port du serveur de test, 5191 par défaut	Non

Les quatre variables S3_ENDPOINT, S3_BUCKET, S3_ACCESS_KEY_ID et S3_SECRET_ACCESS_KEY fonctionnent **ensemble** : si l'une manque, le stockage bascule silencieusement sur le disque local. C'est le piège principal de la configuration.

Variables à prévoir pour l'IA (section 9) : clé du fournisseur, identifiant de modèle, activation par fonctionnalité.
Aucune n'existe aujourd'hui.

13. DÉPLOIEMENT

13.1 Prérequis

- Node.js 22 ou version ultérieure (version utilisée en développement : 22.21.1 ; **aucune version n'est figée dans `package.json`** — il est recommandé d'y ajouter un champ `engines`).
- Accès en lecture au dépôt github.com/harnolmub/dialogue.
- Une chaîne de connexion PostgreSQL.

13.2 Installation locale

```
git clone git@github.com:harnolmub/dialogue.git
cd dialogue
npm install
cp .env.example .env
```

Renseigner ensuite `.env` : au minimum `DATABASE_URL`, `PORT`, `ADMIN_BOOTSTRAP_EMAIL` et `ADMIN_BOOTSTRAP_PASSWORD` (16 caractères minimum). Ce fichier ne doit jamais être versionné.

13.3 Base de données

```
npm run migrate
```

Les migrations sont également appliquées automatiquement au démarrage du serveur. La commande manuelle sert à valider la connexion avant de lancer l'application.

13.4 Lancement

Deux modes :

```
npm run build && npm start
```

sert l'application compilée sur le port choisi — c'est le mode identique à la production, à utiliser pour toute vérification sérieuse.

```
npm run dev
```

lance le serveur de développement Vite avec rechargement à chaud ; les appels `/api` et `/health` sont relayés vers `http://127.0.0.1:5190` (réglable par `VITE_API_TARGET`). Le serveur applicatif doit tourner en parallèle.

13.5 Tests

```
npm run test:e2e
```

Lance 50 tests Playwright sur deux moteurs de rendu. Le lanceur compile le projet et démarre son propre serveur sur le port 5191, avec les limitations de débit relevées pour ne pas se bloquer lui-même. Les données créées pendant les tests sont supprimées automatiquement à la fin de la campagne.

Important : les tests écrivent dans la base désignée par DATABASE_URL. Ne jamais les lancer avec la chaîne de connexion de production.

Couverture : parcours de dépôt public, notes vocales, contrat de l'API, verrous d'accès du backoffice, traitement complet d'une contribution, actions groupées, export, journal d'audit, gestion des comptes, synthèses, structure et rendu responsive des pages publiques.

13.6 Mise en production

```
git checkout -b une-branche-de-travail
# modifications
npm run build
npm run test:e2e
git commit -am "Description de la modification"
git push origin une-branche-de-travail
# fusion dans main après relecture
```

Le déploiement Railway se déclenche ensuite sur la branche configurée (à **confirmer** : main).

13.7 Contrôles après déploiement

Contrôle	Attendu
curl -I https://dialoguenational.cd/	200, en-têtes x-frame-options et permissions-policy présents
curl https://dialoguenational.cd/health	{"ok":true}
curl -I https://dialoguenational.cd/participer	200
curl -I https://dialoguenational.cd/page-inexistante	404
Journaux Railway	Applied 00x_....sql si une migration était en attente, puis CONGO DIALOGUE listening on ...
Dépôt de bout en bout	Déposer une contribution de test, vérifier la référence, la retrouver dans le backoffice, puis la supprimer en base

Repère utile : l'en-tête last-modified de la page d'accueil donne la date du build déployé. Au 15 septembre 2026, il indique le 5 septembre 2026, alors que main contient des commits postérieurs : **la production n'est pas à jour**. Les correctifs concernés incluent l'affichage du drapeau dans le sélecteur de pays et la mise en page de l'en-tête sur tablette.

14. MAINTENANCE

14.1 Déployer une nouvelle version

Voir 13.6, puis 13.7. En cas d'anomalie constatée après la mise en ligne, revenir au déploiement précédent depuis la console Railway (section 3.4) avant de chercher la cause.

14.2 Ajouter une variable d'environnement

1. Ajouter la variable dans la console Railway, sur le service applicatif.
2. Ajouter son **nom** (jamais sa valeur) dans `.env.example` et dans la section 12 de ce mémo.
3. Redéployer : les variables ne sont lues qu'au démarrage du processus.

14.3 Consulter les journaux

Console Railway, service applicatif. Messages émis par l'application :

Message	Signification
CONGO DIALOGUE listening on <port>	Démarrage réussi
Applied 00x_....sql	Migration appliquée
Administrateur initial créé : ...	Compte d'amorçage créé
Contribution DIALOGUE-...-..... enregistrée (thématique)	Dépôt réussi
Contribution submission failed	Échec d'enregistrement, suivi de la trace
Upload failed	Échec de lecture d'un fichier joint
DATABASE_URL absent : ...	Alerte : le service tourne sans base
Database initialization failed	Le service s'est arrêté au démarrage

14.4 Diagnostiquer une erreur 500 au dépôt

1. Rechercher `Contribution submission failed` dans les journaux ; la trace suit.
2. Causes les plus fréquentes, dans l'ordre : base injoignable ; échec d'écriture du fichier (identifiants S3 invalides ou disque plein) ; migration non appliquée.
3. Vérifier la connexion à la base, puis les variables `S3_*`.
4. Reproduire en local avec la même configuration avant de corriger.

14.5 Diagnostiquer une erreur de base

- Vérifier que la base Neon est active et n'est pas en veille.
- Vérifier la validité de `DATABASE_URL` (une rotation de mot de passe la périme, c'est déjà arrivé sur cette plateforme).
- Vérifier que `NODE_ENV=production` est défini : sans cela, le client se connecte sans SSL et Neon refuse la connexion.

- Consulter `SELECT name, applied_at FROM schema_migrations ORDER BY name` pour confirmer l'état du schéma.

14.6 Vérifier le stockage des fichiers

1. Dans le backoffice, ouvrir une contribution ancienne comportant un document.
2. Télécharger le document. S'il est introuvable alors que la fiche le référence, les fichiers ont été perdus : le stockage est sur disque éphémère (section 11.4).
3. Contrôle en base : comparer `SELECT count(*) FROM contribution_files` avec le nombre d'objets réellement présents dans le stockage.

14.7 Vérifier le système de courriels

Sans objet : **aucun envoi de courriel n'est implémenté**. Si un besoin de confirmation au contributeur apparaît, il s'agit d'un développement à part entière, incluant le choix d'un prestataire, la configuration SPF, DKIM et DMARC sur le domaine, et la publication d'enregistrements MX aujourd'hui absents.

14.8 Vérifier l'enregistrement audio

1. Ouvrir /participer en HTTPS, sur un appareil réel.
2. Démarrer un enregistrement, autoriser le microphone, parler : la jauge doit bouger.
3. Terminer, réécouter la prévisualisation.
4. Envoyer, puis réécouter la note depuis le backoffice.
5. En cas d'échec, se reporter au tableau de dépannage de la section 6.4.

Vérification à faire sur au moins un iPhone et un appareil Android : les formats et le comportement de finalisation diffèrent.

14.9 Restaurer après incident

Incident	Action
Mauvais déploiement	Revenir au déploiement antérieur (console Railway)
Données corrompues ou effacées	Restauration Neon à un instant donné — procédure à confirmer et à tester
Fichiers perdus	Aucun recours si le stockage est éphémère et sans sauvegarde
Compte d'administration compromis	Désactiver le compte (les sessions sont révoquées immédiatement), examiner <code>audit_logs</code> , changer les mots de passe

14.10 Sauvegarde

À mettre en place et à documenter : export périodique de la base hors de Neon, sauvegarde du stockage des fichiers, et **test de restauration au moins une fois**. Une sauvegarde jamais restaurée n'est pas une sauvegarde.

15. MONITORING

Aucun outil de supervision n'est installé. La liste ci-dessous indique ce qu'il faut surveiller et par quel moyen, en l'état actuel.

À surveiller	Moyen disponible aujourd'hui	Seuil ou signal d'alerte
Disponibilité	Sonde externe à mettre en place sur /health	Toute réponse différente de {"ok":true}
Temps de réponse	Journaux Railway, mesure manuelle	Page d'accueil au-delà de 3 s
Erreurs serveur	Journaux Railway	Toute occurrence de Contribution submission failed
Base de données	Console Neon	Connexions refusées, latence, quota de stockage
Stockage	Contrôle manuel (14.6)	Écart entre les références en base et les objets présents
Processeur et mémoire	Métriques Railway	Redémarrages répétés du conteneur
Volume de contributions	Tableau de bord du backoffice	Chute brutale : signe d'une panne du formulaire
Dépôts échoués	Journaux	Répétition de Upload failed
Notes vocales échouées	Retours utilisateurs, part des dépôts audio	Baisse soudaine de la part des notes vocales
Courriels	Sans objet	—
Connexions d'administration	Table audit_logs, action LOGIN_FAILED	Séries d'échecs sur un même compte
Erreurs d'IA	Sans objet aujourd'hui	À prévoir avec la section 9

Priorité recommandée : une sonde externe sur /health avec alerte, et une alerte sur les échecs de connexion répétés. Ce sont les deux angles morts les plus dangereux aujourd'hui.

Rappel : /health ne teste pas la base. Une sonde plus utile interrogerait une route vérifiant une requête simple en base — **amélioration recommandée**.

16. RESPONSABILITÉS

Matrice des rôles à pourvoir. Un même individu peut couvrir plusieurs lignes, mais chaque ligne doit avoir un titulaire identifié.

Domaine	Profil recommandé	Périmètre
Infrastructure	Administrateur système / DevOps	Compte Railway, déploiements, variables, volumes, retours arrière
Développement	Développeur web JavaScript	Dépôt Git, correctifs, évolutions, tests
Base de données	Administrateur de base de données	Neon, migrations, sauvegardes, restauration
Sécurité	Responsable sécurité	Accès, rotation des secrets, revue du journal d'audit, incidents
Administration fonctionnelle	Super administrateur de la plateforme	Comptes du backoffice, rôles, habilitations
Documents officiels	Chargé de communication institutionnelle	Vérification, publication et versionnement des documents
Contributions	Équipe de traitement	Recevabilité, classification, affectation
Analyse	Analystes thématiques	Lecture, transcription, synthèses
Intelligence artificielle	Responsable technique IA	Architecture, coûts, qualité, garde-fous, validation humaine
Communication	Direction de la communication	Discours public, mobilisation, réponses aux citoyens
DNS et domaine	Administrateur système	Cloudflare, renouvellement du domaine .cd

17. PLAN DE CONTINUITÉ

17.1 Railway indisponible

Le site est hors ligne : aucun dépôt n'est possible. Mesures : vérifier l'état du service Railway, publier un message d'information par les canaux de communication, et si l'indisponibilité se prolonge, redéployer l'application chez un autre hébergeur — l'application est un serveur Node.js standard sans dépendance propriétaire, donc portable. Prérequis : disposer de la chaîne de connexion à la base et des variables d'environnement.

17.2 Base de données indisponible

Le site répond mais **aucune contribution ne peut être enregistrée** : le dépôt renvoie une erreur 500. Le backoffice est inutilisable. Mesures : vérifier l'état de Neon, la validité de la chaîne de connexion, et restaurer si nécessaire. Ne pas redémarrer le service sans DATABASE_URL valide : il basculerait silencieusement en écriture fichier, créant des contributions hors base à réconcilier ensuite manuellement.

17.3 Stockage indisponible

Les dépôts comportant un fichier ou une note vocale échouent ; les dépôts en texte seul continuent de fonctionner. Les fiches existantes affichent leurs fichiers comme introuvables. Mesure d'urgence : inviter les contributeurs à déposer en texte pendant l'incident.

17.4 Domaine hors service

Vérifier l'expiration du domaine .cd et les serveurs de noms chez le bureau d'enregistrement. L'application reste joignable par l'adresse fournie par Railway (**à confirmer**), qui peut servir d'accès de secours pour les équipes le temps du rétablissement.

17.5 Problème Cloudflare

Si le proxy pose problème, il est possible de le désactiver temporairement (passage en DNS seul) ; le TLS est alors assuré par Railway. Mesure à prendre en connaissance de cause : la protection contre les attaques par déni de service disparaît.

17.6 Courriels

Sans objet aujourd'hui. Aucune fonctionnalité ne dépend du courriel : une panne de messagerie n'a aucun effet sur la plateforme.

17.7 IA indisponible

Aucun effet sur la réception des contributions, aujourd'hui comme demain. C'est une exigence d'architecture : les traitements d'IA doivent être asynchrones et facultatifs, l'interface doit fonctionner sans eux, et l'absence de suggestion automatique ne doit jamais empêcher un analyste de traiter une contribution à la main.

18. CHECKLIST DE REPRISE DU PROJET

À parcourir avec le détenteur actuel des accès. Aucun identifiant ni secret ne doit être consigné dans ce document : seule la confirmation de la remise est cochée.

Accès

- ☐ Dépôt Git `github.com/harnolmub/dialogue` — droits d'écriture sur main
- ☐ Console Railway — projet et service de l'application
- ☐ Console Neon — bases de développement et de production
- ☐ Console Cloudflare — zone `dialoguenational.cd`
- ☐ Bureau d'enregistrement du domaine `.cd` — titulaire et échéance
- ☐ Stockage des fichiers — console du fournisseur S3, si le stockage objet est utilisé
- ☐ Compte super administrateur du backoffice sur `/admin`
- ☐ Outil de supervision — sans objet aujourd'hui, à créer

Configuration

- ☐ Liste complète des variables d'environnement définies sur le service Railway
- ☐ Confirmation de la branche qui déclenche le déploiement
- ☐ Confirmation de la commande de compilation exécutée au déploiement
- ☐ Confirmation du mode de stockage des fichiers : objet S3 ou disque, et si disque, existence d'un volume persistant
- ☐ Chaîne de connexion de production valide et testée

Exploitation

- ☐ Procédure de sauvegarde de la base — définie, documentée, planifiée
- ☐ Procédure de restauration — **testée au moins une fois**
- ☐ Procédure de sauvegarde des fichiers déposés
- ☐ Procédure de retour arrière d'un déploiement, testée
- ☐ Mot de passe du compte d'amorçage changé après la première connexion
- ☐ Liste nominative des comptes du backoffice, avec leur rôle, revue et validée

Documentation

- ☐ Ce mémo, à jour
- ☐ Matrice des responsabilités renseignée avec des noms
- ☐ Points marqués « À confirmer » levés un par un

ANNEXE A — Récapitulatif des points « À confirmer »

Section	Point à confirmer
2.6, 3.2, 11.4	Mode de stockage des fichiers en production et existence d'un volume persistant
3.2	Nom du projet et du service Railway, commandes configurées, version de Node.js, branche de déploiement, variables définies, environnement de préproduction
3.5	Bureau d'enregistrement du domaine, échéance, titulaire
3.6	Réglages Cloudflare : mode TLS, HSTS, règles de redirection, pare-feu applicatif
4.1	Chaîne de connexion de production courante (celle transmise n'est plus valide)
4.6	Rétention de l'historique Neon, procédure de restauration, exports hors Neon
11.3	Chiffrement au repos des données côté Neon
13.1	Version de Node.js exécutée en production
17.4	Adresse de secours fournie par Railway

ANNEXE B — Commandes de référence

Besoin	Commande
Installer	<code>npm install</code>
Compiler	<code>npm run build</code>
Démarrer comme en production	<code>npm start</code>
Développer avec rechargement	<code>npm run dev</code>
Appliquer les migrations	<code>npm run migrate</code>
Lancer les tests	<code>npm run test:e2e</code>
Tests avec interface	<code>npm run test:e2e:ui</code>
Vérifier la santé en ligne	<code>curl https://dialoguenational.cd/health</code>

ANNEXE C — Fichiers structurants

Fichier	Rôle
server.js	Serveur Express, dépôt public, service des pages, démarrage
server/db.js	Pool PostgreSQL, migrations automatiques, enregistrement d'une contribution
server/storage.js	Stockage des fichiers : S3 ou disque, lecture privée
server/admin-auth.js	Mots de passe, sessions, permissions, journal d'audit, compte d'amorçage
server/admin-api.js	Les 25 routes du backoffice
src/main.js	Page d'accueil : navigation, assistant guidé, modale, animations
src/participer.js	Formulaire, sélecteur de pays, enregistreur vocal, envoi
src/admin.js	Application du backoffice
migrations/*.sql	Schéma de la base, par ordre d'application
vite.config.js	Compilation multi-pages, proxy de développement
playwright.config.js	Tests : serveur dédié, deux moteurs, nettoyage final
tests/e2e/	50 tests de bout en bout

Fin du mémo technique — Dialogue national, version 1.0, 15 septembre 2026.